

Gated Recurrent Transformers: Expressive Depth through Recurrent Modulation

Amr Hegazy

The German University in Cairo
amr.hazem@student.guc.edu.eg

Amr Alanwar

Technical University of Munich
alanwar@tum.de

Mostafa Elhoushi

Cerebras Systems Inc.
mostafa.elhoushi@cerebras.net

Abstract

Scaling transformer language models creates an inherent tension between expressivity and memory efficiency. While unique weights across layers preserve functional specialization—from input-grounding to abstract refinement—they incur a substantial memory footprint. Conversely, standard depth-sharing enforces uniform transformations that collapse representational diversity and degrade modeling quality. We introduce the GATED RECURRENT TRANSFORMER (GRT), a recurrent depth transformer where fixed-depth prelude and coda blocks bracket a single shared core iterated R times. Inspired by gated recurrent neural networks, we employ a lightweight projection and an elementwise update gate—conditioned on the hidden state, the fixed prelude output, and noise resampled at every step—to modulate the recurrent update. This allows the model to specialize the input to the same few layers across recurrences, rather than requiring many unique layers to achieve functional diversity. Under an *iso*FLOPs constraint, a 3-layer GATED RECURRENT TRANSFORMER matches the accuracy of a 12-layer GPT-2 Small baseline with similar training and inference FLOPs, and leads MoR and heavy-tail depth sampling in all nine scale-by-budget cells; at medium and large scale it approaches dense quality at the standard token budget and overtakes it at medium scale once that budget is doubled. Under an *iso*PARAMS constraint, deeper recurrence achieves a **2.76** validation loss versus **2.84** for a non-recurrent counterpart at matched parameter and data budget. Our results demonstrate that adaptive depth reuse is a principled strategy for trading parameters for quality: at large scale, 62% fewer parameters and 59% less peak decoding memory for a 10% increase in compiled generation latency. Code is available at <https://github.com/Amr-Hegazy1/gated-recurrent-transformer>.

1 Introduction

Scaling transformer language models has delivered remarkable gains, yet it necessitates a simultaneous increase in parameter count and training compute [1]. In standard architectures, depth and parameters are rigidly coupled: every additional layer introduces a fresh set of weights, creating a memory bottleneck that constrains effective depth under fixed hardware budgets.

Beyond memory efficiency, however, there is a deeper computational motivation rooted in the nature of intelligence itself: Turing’s foundational insight [2] is that a *finite* set of states and symbols, applied *iteratively*, is sufficient to compute anything computable — suggesting that the power of a reasoning system lies not in its breadth of parameters, but in its depth of iteration. This principle finds a striking

modern echo in the test-time compute paradigm [3], where models allocate more compute at inference to improve reasoning [4, 5]. While existing approaches realize this compute through chain-of-thought steps in the output space—consuming sequence length in the process [6]—recurrent depth offers a complementary realization: by iterating a shared transformation over the input representation, the model “thinks” more deeply entirely within its hidden states without generating extra tokens or storing extra parameters.

Weight sharing is the natural response to memory-depth coupling, allowing effective depth to grow without inflating parameter count [7–10]. Prior work has utilized recurrent depth either as an efficiency tool to match quality at reduced parameter counts [11, 12] or as a performance tool to improve accuracy at matched parameter counts [9, 10]. However, a unified treatment of these perspectives is missing, and a deeper architectural tension remains: standard weight sharing forces an identical transformation on an ever-evolving representation. Because the hidden state after the first recurrence differs fundamentally from the eighth, a static transformation collapses the functional diversity that makes depth valuable [13]. This raises the question: Can a single fixed transformation serve meaningfully across such diverse representational stages, or does it inevitably collapse the functional diversity that makes depth valuable in the first place [13]. Such questions motivate the design of GATED RECURRENT TRANSFORMER.

To resolve this tension, GATED RECURRENT TRANSFORMER introduces a per-element learned gate that conditions on the current hidden state, a fixed *prelude* representation of the original input [9], and stochastic noise. This constructs a distinct, context-grounded input at every recurrence, allowing a single weight tensor to behave as multiple specialized layers. Gates are initialized such that training starts with the residual stream passing through nearly unchanged through all recurrences. As training progresses, gating emerges gradually as the shared block learns to selectively refine representations. The shared transformation is thus fixed in weights but dynamic in behavior — the model thinks differently at each pass despite reusing the same parameters. Combined with depth sampling during training [9], this architecture enables a continuous compute-quality tradeoff at inference and facilitates evaluation across both *iso*FLOPS and *iso*PARAMS regimes.

We summarize our contributions as follows:

- **Architecture:** We propose GATED RECURRENT TRANSFORMER, which uses per-element gating and stochastic perturbations to prevent representational collapse, allowing a shared core to behave distinctly across recurrence steps.
- **Unified Evaluation:** We evaluate GATED RECURRENT TRANSFORMER under both *iso*FLOPS and *iso*PARAMS regimes at three scales. Under *iso*FLOPS it leads MoR and heavy-tail depth sampling in all nine scale-by-budget cells and matches or leads RRT and Ouro at every scale at the standard budget, matches the dense GPT-2 baseline at small scale while using **64%** fewer parameters, and closes on it at medium and large scale as the token budget grows. Under *iso*PARAMS it improves validation loss at matched parameter count, at a higher inference-FLOP cost.
- **Emergent Early Exit:** Without auxiliary losses, GATED RECURRENT TRANSFORMER naturally supports inference-time early exiting, retaining **~92%** accuracy when running only half of its recurrence steps.

2 Related Work

The concept of shared weights across repeated computational steps traces back to the seminal 1986 backpropagation paper of Rumelhart, Hinton, and Williams[14], who briefly proposed at the end of their paper the synchronous iterative net: a network where each iteration corresponds to a layer with tied weights. [15] extended this to temporal sequence modeling via backpropagation through time (BPTT), inspiring recurrent architectures — RNNs [16], LSTMs [17], and GRUs [18] — that apply shared weights along the sequence dimension, processing inputs one timestep at a time. In contrast, this work applies recurrent depth across the full input, iterating a shared-weight transformation along network depth to progressively refine the input representation.

In the era of transformers, one of the earliest papers on recurrent depth was ALBERT [7], which proposed a BERT [19] model with all transformer layers sharing the same weights. Universal Transformers [8] further added a per-token adaptive halting mechanism. However, both methods shared the *entire* layer stack, forcing every layer to apply an identical transformation regardless

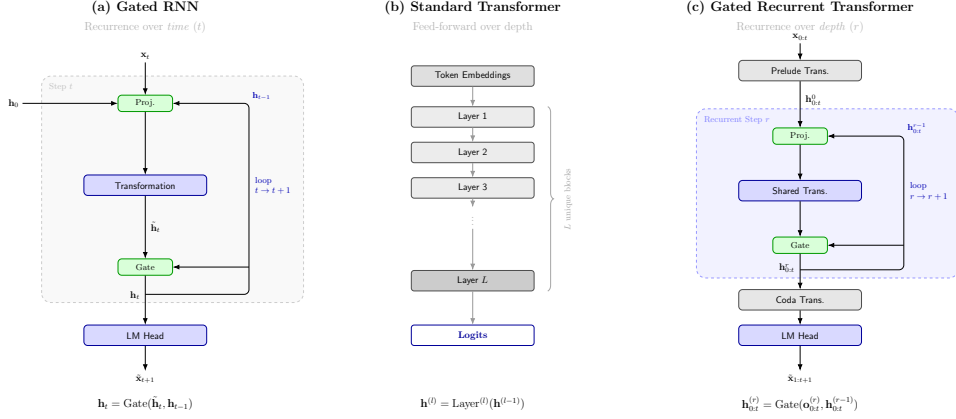


Figure 1: **GATED RECURRENT TRANSFORMER overview.** (a) A Gated Recurrent Unit (GRU) reuses the same weight matrices at every sequence position—recurrence over the *sequence* dimension. (b) A standard transformer stacks L independent layers, each with unique weights, applied once to the full sequence. (c) GATED RECURRENT TRANSFORMER introduces recurrence over the *depth* dimension, applying each recurrence to the full sequence

of the representational stage of the input. [9] relaxed this constraint by assigning distinct roles to prelude, shared core, and coda layers. Similarly, Koishikenov et al. [20] identified early, middle, and late layers of pretrained LLMs as encoding, reasoning, and decoding stages respectively, and proposed recurring only the middle layers while keeping the rest fixed. Our GATED RECURRENT TRANSFORMER follows this principle of selectively recurring middle layers.

Prior work has shown that parameters in feed-forward network (FFN) layers primarily store factual knowledge [21], while computational depth drives reasoning ability [22], motivating recurrence to improve reasoning. We organize prior work on recurrent depth transformers along two evaluation regimes. In the *isoPARAMS* regime — increasing FLOPs by recurring layers while keeping parameter count fixed, trading compute for improved accuracy — [9, 10, 20] demonstrate consistent quality gains. In the *isoFLOPs* regime — reducing parameter count by recurring layers while matching training and inference compute, trading memory for efficiency — [11] and [12] show that depth reuse can achieve comparable quality at a smaller memory footprint. Our work spans both regimes, with *isoFLOPs* as our primary contribution and complementary *isoPARAMS* results reported in Section 4.

A related line of work adapts the amount of computation dynamically rather than fixing it uniformly across tokens or layers. Adaptive Computation Time (ACT) [23] halts computation per-position based on learned termination signals, allowing different tokens to consume different amounts of compute. Mixture-of-Depths [24] takes a routing approach, dynamically assigning tokens to different subsets of layers rather than processing all tokens through all layers. More recently, Mixture-of-Recursions [25] extends this idea to the recurrent setting, routing subsets of tokens to different numbers of recursion steps. While all three methods adapt computation at the token level — through halting, layer routing, or recursion routing — they require dedicated termination signals or routing mechanisms. In contrast, GATED RECURRENT TRANSFORMER uses a simpler gating mechanism that modulates how much of the shared-block update is absorbed at each recurrence step, without any routing or halting logic.

Weight-tied iteration toward a fixed point is the defining feature of deep equilibrium models [26], which solve for the equilibrium directly and differentiate through it with implicit gradients, avoiding the memory cost of storing an unrolled trajectory. GATED RECURRENT TRANSFORMER shares the weight-tied update but not the equilibrium objective: we unroll a fixed number of discrete steps and backpropagate through them, and depth sampling trains each step to serve as an exit rather than as an approach to a single limit.

Scaling laws [1, 27] have established how model size and data jointly determine performance, with Pythia [28] providing controlled comparisons across scales. Notably, Kaplan et al. [1] directly evaluated parameter-sharing transformers and observed that recurrent models perform better at matched parameter count (*isoPARAMS*) but worse at matched compute (*isoFLOPs*). The interplay between

depth and width has received dedicated attention: [29] theoretically shows that increasing width can compensate for reduced depth in algorithmic reasoning tasks, while [30] empirically demonstrates that scaling law prescriptions are sensitive to depth-width ratio. In our GATED RECURRENT TRANSFORMER, we propose a novel recurrent depth architecture that aims to improve the scaling laws of loss versus parameter count and loss versus training FLOPs, demonstrating consistent gains under both *iso*FLOPs and *iso*PARAMS regimes as model size and training data scale.

3 Recurrent Depth Reuse for Language Modeling

3.1 Preliminaries

Let $\mathbf{X} = (x_1, \dots, x_T)$ be a token sequence of length T drawn from vocabulary $\mathcal{V}$. A standard autoregressive transformer with L blocks defines a chain of residual updates: $\mathbf{h}^{(\ell)} = \mathbf{h}^{(\ell-1)} + \text{Block}_\ell(\mathbf{h}^{(\ell-1)})$ for $\ell = 1, \dots, L$, where d is the embedding dimension and $\mathbf{h}^{(0)} \in \mathbb{R}^{T \times d}$ are initial token embeddings. Each block comprises pre-norm multi-head self-attention and a token-wise MLP. The objective is to minimize the negative log-likelihood, $\mathcal{L} = -\sum_t \log p(x_t | x_{<t})$. Standard transformers couple depth and parameters rigidly, as every additional layer adds a fresh set of weights. The total unique parameter count scales as $\Theta(L \cdot d^2)$.

3.2 The Prelude–Shared–Coda Architecture

GATED RECURRENT TRANSFORMER follows [9] in partitioning transformer blocks into three sets, denoted by the shorthand $n_{\text{pre}} + n_{\text{rec}} \times R + n_{\text{coda}}$. We have observed that this separation of fixed context encoders from a shared recurrent core yields more stable training than uniform weight sharing across all layers.

- **Prelude (n_{pre} blocks):** Applied once to produce a stable, context-aware conditioning signal $\mathbf{h}^{(\text{pre})} = \text{Block}_{n_{\text{pre}}} \circ \dots \circ \text{Block}_1(\mathbf{h}^{(0)})$.
- **Shared core (n_{rec} blocks):** Applied recurrently R times. The first step is initialized with stochastic noise ϵ_x conditioned on the prelude output. In subsequent steps, the core iteratively processes the previous hidden state $\mathbf{h}^{(r-1)}$ alongside the fixed prelude anchor: $\mathbf{h}^{(R)} = \text{Block}_{n_{\text{shared}}} \circ \dots \circ \text{Block}_1[\epsilon_x, \mathbf{h}^{(\text{pre})}]$. We provide more details on the operation in the subsequent section.
- **Coda (n_{coda} blocks):** Receives the final refined state $\mathbf{h}^{(R)}$ and projects to output final hidden state $\mathbf{h}^{(\text{coda})} = \text{Block}_{n_{\text{coda}}} \circ \dots \circ \text{Block}_1(\mathbf{h}^{(R)})$ that are then fed into the language model to produce logits.

Total unique parameter count is $\Theta((n_{\text{pre}} + n_{\text{rec}} + n_{\text{coda}}) \cdot d^2)$, independent of R . A configuration such as $2+5 \times 4+2$ visits $2 + 5 \cdot 4 + 2 = 24$ block executions per forward pass while storing weights for only $2 + 5 + 2 = 9$ blocks— $2.6 \times$ fewer unique blocks than the 24-layer GPT-2 medium baseline it is isoFLOP-matched to. Figure 2 illustrates this layout.

Per-token forward-pass cost for a $n_{\text{pre}} + n_{\text{rec}} \times R + n_{\text{coda}}$ configuration at sequence length S is

$$\text{FLOPs} = (n_{\text{pre}} + n_{\text{rec}}R + n_{\text{coda}})(24d^2 + 4Sd) + R \cdot 10d^2, \quad (1)$$

where $24d^2 + 4Sd$ is the standard per-block cost (attention and MLP projections plus the two attention matmuls) and $10d^2$ is the per-step overhead of the recurrent projection W_{proj} and the gate MLP f_g .

3.3 Recurrent Projection and Elementwise Gate

Merely feeding the output of each recurrence as the input to the next recurrence risks forgetting the original input context, suffers from vanishing or exploding gradients across deep recurrences, and taxes the representational capacity of a single shared weight tensor.

Inspired by RNNs [16] in general, and GRUs [18] in particular, we treat the hidden representation after each recurrence step as a *state* that evolves across recurrences (as illustrated in Figure 1). Each application of the shared core reads this state, transforms it, and writes an updated state back—analagous to an RNN cell, but operating over depth rather than time. This framing motivates the gating mechanism below: just as GRU gates control how much of its cell state is overwritten at each

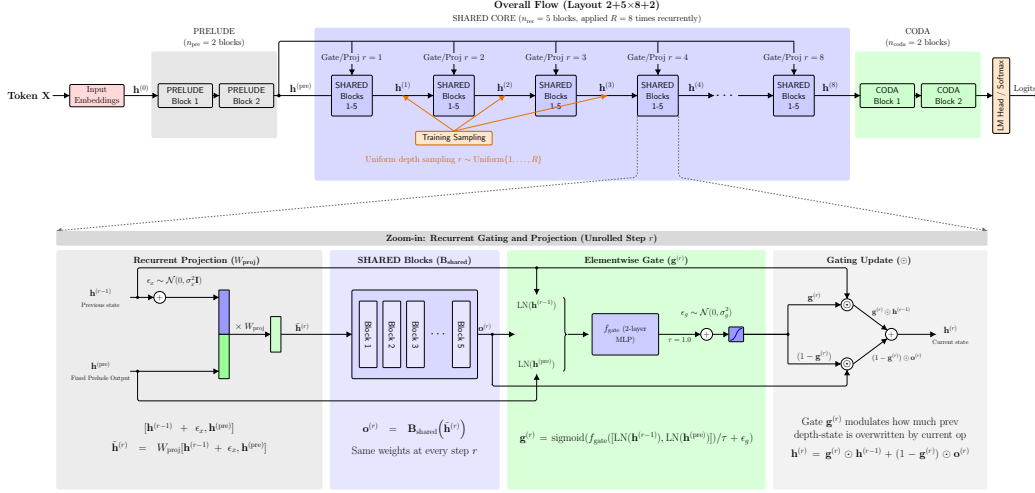


Figure 2: **GATED RECURRENT TRANSFORMER architecture (2+5×8+2 layout).** *Top:* Two prelude blocks (gray) encode a fixed context representation $\mathbf{h}^{(\text{pre})}$ that is held constant across all recurrence steps. Five shared blocks (blue) are applied $R = 8$ times recurrently; $\mathbf{h}^{(\text{pre})}$ conditions the projection and gate at each step. Two coda blocks (green) project to logits. *Bottom:* Zoom-in of a single recurrence step r : the current state $\mathbf{h}^{(r-1)}$ is projected together with $\mathbf{h}^{(\text{pre})}$ via W_{proj} , processed by the shared blocks to produce proposal $\mathbf{o}^{(r)}$, and blended back via the elementwise gate $\mathbf{g}^{(r)} \in [0, 1]^{T \times d}$.

timestep, GATED RECURRENT TRANSFORMER’s elementwise gate controls how much of the current depth-state is replaced by the shared block’s output.

Hence, we introduce our novel adaptive gating approach that aims to mitigate the 3 aforementioned problems. At each recurrence step r , GATED RECURRENT TRANSFORMER prepares the input to $\mathcal{B}_{\text{shared}}$ by projecting the current state together with the fixed prelude output, then selectively writes the block’s output back to the residual stream via a learned elementwise gate. The full update is:

$$\tilde{\mathbf{h}}^{(r)} = W_{\text{proj}} [\mathbf{h}^{(r-1)} + \epsilon_x, \mathbf{h}^{(\text{pre})}], \quad (2)$$

$$\mathbf{o}^{(r)} = \mathcal{B}_{\text{shared}}(\tilde{\mathbf{h}}^{(r)}), \quad (3)$$

$$\mathbf{h}^{(r)} = \mathbf{g}^{(r)} \odot \mathbf{h}^{(r-1)} + (1 - \mathbf{g}^{(r)}) \odot \mathbf{o}^{(r)}, \quad (4)$$

where $[\cdot, \cdot]$ denotes concatenation along the feature axis, $\mathbf{g}^{(r)} \in [0, 1]^{T \times d}$ is a learned elementwise gate (defined in full below in Eq. (5)), $W_{\text{proj}} \in \mathbb{R}^{d \times 2d}$ is a learned projection, $\epsilon_x \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ is injected additive noise, and $\odot$ is elementwise multiplication.

The gate $\mathbf{g}^{(r)} \in [0, 1]^{T \times d}$ is produced by a small feed-forward network conditioned on the normalised current state and the normalised prelude output:

$$\mathbf{g}^{(r)} = \sigma \left(f_{\mathbf{g}}([\text{LN}(\mathbf{h}^{(r-1)}), \text{LN}(\mathbf{h}^{(\text{pre})})]) / \tau + \epsilon_g \right), \quad (5)$$

where $\sigma(\cdot) = 1/(1 + e^{-\cdot})$ is the sigmoid function, LN denotes layer normalization [31], τ is a temperature hyperparameter (set to 1.0 throughout our experiments), $\epsilon_g \sim \mathcal{N}(0, \sigma_g^2)$ is per-scalar gate noise injected during training, and $f_{\mathbf{g}}$ is a two-layer MLP with SiLU activation and hidden dimension $d_{\text{gate}} = d$. The second linear layer of $f_{\mathbf{g}}$ has its bias initialised to +4, placing $\mathbf{g}^{(r)} \approx 0.98$ at the start of training: the copy branch of Eq. (4) dominates, the residual stream passes through the recurrence almost unchanged, and the model learns which elements to overwrite as training proceeds. Positive initialisation of a forget gate is long-standing practice in recurrent networks [32], and Jozefowicz et al. [33] found a bias of +1 or +2 enough to bring a vanilla LSTM level with the best variants in their search. We simply apply the same principle over depth rather than time. We swept across $\{-2, 0, +2, +4\}$ at the full training horizon the spread is 0.019 nats (Section B.6).

By defining $\mathbf{g}_{t,i}^{(r)}$ as the i^{th} element of the gate for token t at recurrence r , GATED RECURRENT TRANSFORMER achieves a highly granular, per-element specialization. When $\mathbf{g}_{t,i}^{(r)} \rightarrow \mathbf{1}$, the residual stream remains unchanged, whereas $\mathbf{g}_{t,i}^{(r)} \rightarrow \mathbf{0}$ allows the block output to fully replace the state. This per-recurrence specialization provides a more memory-efficient alternative to approaches such as Per-Layer Embeddings (PLE) (used in recent models like Gemma3n [34] and Gemma-4 [35]). While PLE requires L unique embedding layers to provide layer-specific context, our approach conditions the gate on $\mathbf{h}^{(\text{pre})}$ to provide a constant view of the *original* input at every step, while modulating it with $\mathbf{h}^{r-1}$ and stochasticity ϵ_g .

State noise ϵ_x (Eq. (2)) perturbs the recurrent projection input, discouraging the model from learning brittle exact-match patterns across steps. Gate noise ϵ_g (Eq. (5)) prevents the gate from collapsing to a near-constant value during training. Specific noise magnitudes (σ_x, σ_g) are reported in Section 4.

3.4 Recurrence Depth Sampling During Training

At each training step, we sample $r \sim \text{Uniform}\{1, \dots, R\}$. This serves two purposes: (1) it implicitly trains every exit point, enabling **early exit at inference without auxiliary loss terms** or the associated gradient interference common in multi-exit models [36, 37]; and (2) it acts as stochastic depth regularization, improving final validation loss over fixed-depth schedules (as will be shown later in our ablations Section 5).

We summarize the forward pass of GATED RECURRENT TRANSFORMER in Listing 1.

4 Experiments & Results

Experimental Setup We build on the nanoGPT codebase [38] for our transformer backbone implementation. All models are trained on a diverse dataset of text with sequence length $T = 1024$ tokens and GPT-2 BPE tokenisation (50,257-token vocabulary). We use AdamW [39] with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, a 2,000-step linear learning rate warmup, and a cosine learning-rate schedule from a peak of 6×10^{-4} to 6×10^{-5} .

Gradient norms are clipped at 1.0. All runs used bfloat16 mixed precision. State noise $\sigma_x = 0.1$ and gate noise $\sigma_g = 0.1$ are used throughout all GATED RECURRENT TRANSFORMER runs; the gate temperature is fixed at $\tau = 1.0$.

We train for 20,000-steps and use an effective batch size of $\approx 491,520$ tokens per step (batch size 8, gradient accumulation 60, sequence length 1024), totalling approximately **9.8B** tokens per run. Dense baselines (GPT-2 small, medium, and large [40]) are trained from scratch under the identical setup; no pre-trained weights are used. We train four recurrent competitors from scratch under the same recipe: *Mixture-of-Recursions* (MoR) [25]; a variant following Geiping et al. [9] that uses the prelude-coda layout with heavy-tail Poisson depth sampling; *Relaxed Recursive Transformers* (RRT) [12], which relaxes weight tying with per-recurrence LoRA adapters; and *Ouro* [10], which supervises every loop iteration.

Main Results Table 1 summarizes performance across model sizes. In the *isoFLOPs* regime, GATED RECURRENT TRANSFORMER achieves competitive quality while storing only **36–37%** of the baseline’s parameters. At small scale it reaches lower loss than the dense baseline on all three seeds we ran: 3.145 ± 0.004 against 3.188 ± 0.056 , our worst seed at 3.148 and the baseline’s best at 3.154 (Appendix B.1). At medium and large scale the dense baseline leads at the standard budget, by 0.05 and 0.06 nats respectively, and Section 5 shows both gaps closing as the token budget grows.

Against the recurrent baselines the margin widens with scale. RRT is the strongest of the four and reaches parity at small scale, 3.143 against our 3.141, a difference well inside the ± 0.004 seed spread; at medium it trails by 0.06 nats and at large by 0.08. MoR and heavy-tail Poisson trail by 0.08 to 0.16 nats at every scale. We attribute the widening gap to what each method holds fixed. RRT’s LoRA deltas are chosen at training time and applied identically to every input, so the diversity they buy is fixed in advance and does not grow with the number of recurrences, whereas the gate conditions on the state it is about to update and keeps successive states separable where uniform sharing collapses them. Ouro is competitive at small scale (3.19) but its medium run converged to 2.93 and its large run converged to 2.82. Strict weight tying is also what leaves R^* free at inference and produces the

Table 1: **Main results (validation loss).** *Upper: isoFLOPs regime*—GATED RECURRENT TRANSFORMER matches forward-pass FLOPs of the dense baseline with fewer unique parameters. *Shaded rows* are dense baselines. *Lower: isoPARAMS regime*—matched unique parameters, higher FLOPs per step. **Layers** = unique transformer blocks. ↓ lower is better. †RRT adds one LoRA adapter per recurrence, so its weights are not identical across steps.

Model	Config	Layers	FLOPs/fwd	Params	Val Loss ↓
<i>isoFLOPs regime (matched FLOPs per forward pass)</i>					
<i>GPT-2 Small</i>					
Baseline	12L	12	1.84 G	124M	3.15
MoR [25]	1+1×10+1	3	1.84 G	45M	3.30
Heavy-tail Poisson [9]	1+1×10+1	3	1.84 G	34M	3.23
Ouro [10]	1+1×10+1	3	1.84 G	35M	3.19
RRT [12]†	1+1×10+1	3	1.84 G	33M	3.14
GRT (ours)	1+1×10+1	3	1.84 G	35M	3.14
<i>GPT-2 Medium</i>					
Baseline	24L	24	7.35 G	354M	2.84
MoR [25]	2+5×4+2	9	7.35 G	137M	3.02
Heavy-tail Poisson [9]	2+5×4+2	9	7.35 G	124M	2.97
Ouro [10]	2+5×4+2	9	7.35 G	126M	2.93
RRT [12]†	2+5×4+2	9	7.35 G	125M	2.95
GRT (ours)	2+5×4+2	9	7.35 G	127M	2.89
<i>GPT-2 Large</i>					
Baseline	36L	36	21.1 G	774M	2.71
MoR [25]	1+5×6+5	11	21.1 G	309M	2.91
Heavy-tail Poisson [9]	1+5×6+5	11	21.1 G	291M	2.93
Ouro [10]	1+5×6+5	11	21.1 G	290M	2.82
RRT [12]†	1+5×6+5	11	21.1 G	289M	2.85
GRT (ours)	1+5×6+5	11	21.1 G	293M	2.77
<i>isoPARAMS regime (matched unique parameter count)</i>					
<i>GPT-2 Small</i>					
Baseline	12L	12	1.84 G	124M	3.15
MoR [25]	1+10×10+1	12	15.64 G	135M	3.12
Heavy-tail Poisson [9]	1+10×10+1	12	15.64 G	124M	3.06
Ouro [10]	1+10×10+1	12	15.64 G	127M	3.10
RRT [12]†	1+10×10+1	12	15.64 G	125M	3.08
GRT (ours)	1+10×10+1	12	15.64 G	127M	3.04
<i>GPT-2 Medium</i>					
Baseline	24L	24	7.35 G	354M	2.84
MoR [25]	2+20×4+2	24	26.4 G	367M	2.78
Heavy-tail Poisson [9]	2+20×4+2	24	26.4 G	356M	2.74
GRT (ours)	2+20×4+2	24	26.4 G	357M	2.76
<i>GPT-2 Large</i>					
Baseline	36L	36	21.1 G	774M	2.71
MoR [25]	3+30×6+3	36	109.0 G	795M	2.69
Heavy-tail Poisson [9]	3+30×6+3	36	109.0 G	776M	2.70
GRT (ours)	3+30×6+3	36	109.0 G	779M	2.65

early-exit curve of Figure 3a, which a fixed set of adapters cannot produce from a single checkpoint. Appendix B.3 tabulates the design differences alongside training cost.

In the *isoPARAMS regime*, increasing recurrence R at a fixed parameter budget yields significant gains: a **0.08** nat improvement at Medium scale and **0.06** at Large scale. This demonstrates that for a fixed memory footprint, trading inference FLOPs for recurrent depth is a principled strategy for improving model expressivity.

Table 2 reports accuracy across nine benchmarks at large scale using `lm-eval-harness` [41]. The *isoFLOPs* variant matches the dense model on average (42.08 vs. 42.05), confirming that parameter

Table 2: **Downstream task evaluation (large scale)**. Zero-shot accuracy on standard benchmarks using `lm-eval-harness`. $\uparrow$ higher is better; deltas relative to GPT-2 Large. **Orange**: best *iso*FLOPs result; **blue**: best *iso*PARAMS result. The *iso*PARAMS model outperforms the dense baseline on average by **+2.10 points**; the *iso*FLOPs model matches GPT-2 Large at only 37% of its parameter count.

Task	GRT <i>iso</i> FLOPs 1+5×6+5 288M, 11L (−63%)	GPT-2 Large (baseline) 774M, 36L	GRT <i>iso</i> PARAMS 3+30×6+3 774M, 36L
ARC-Challenge ($\uparrow$)	25.43 $\uparrow$ 1.80	23.63	25.26 $\uparrow$ 1.63
ARC-Easy ($\uparrow$)	42.89 $\downarrow$ 0.42	43.31	45.54 $\uparrow$ 2.23
BoolQ ($\uparrow$)	60.18 $\uparrow$ 2.60	57.58	52.29 $\downarrow$ 5.29
HellaSwag ($\uparrow$)	35.62 $\downarrow$ 1.59	37.21	41.52 $\uparrow$ 4.31
LAMBADA (OpenAI) ($\uparrow$)	39.05 $\downarrow$ 1.00	40.05	45.27 $\uparrow$ 5.22
LAMBADA (Standard) ($\uparrow$)	30.60 $\uparrow$ 0.87	29.73	38.02 $\uparrow$ 8.29
OpenBookQA ($\uparrow$)	28.40 $\downarrow$ 1.40	29.80	30.20 $\uparrow$ 0.40
PIQA ($\uparrow$)	64.31 $\downarrow$ 1.20	65.51	66.05 $\uparrow$ 0.54
Winogrande ($\uparrow$)	52.25 $\uparrow$ 0.63	51.62	53.28 $\uparrow$ 1.66
Average ($\uparrow$)	42.08 $\uparrow$ 0.03	42.05	44.15 $\uparrow$ 2.10

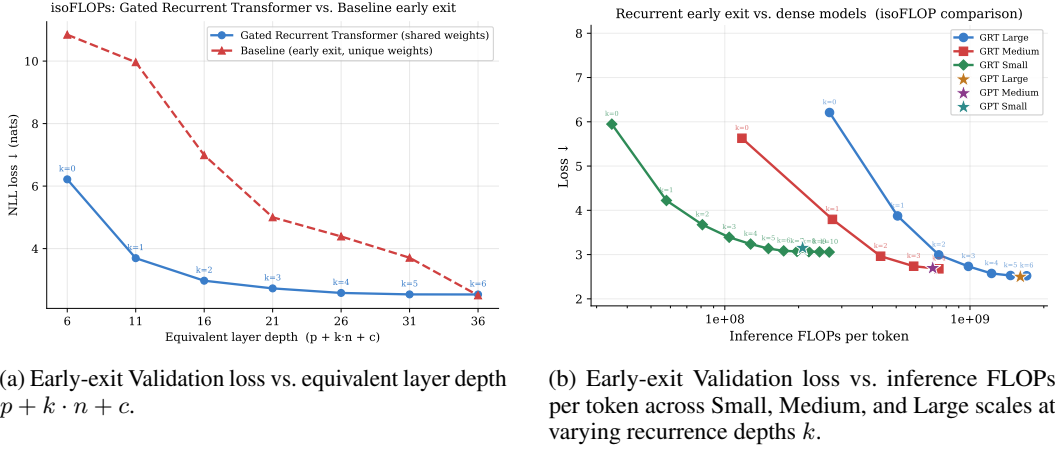


Figure 3: **Early-Exit Analysis** Inference FLOPs are measured based on sequence length 1024, batch size 1.

efficiency translates across evaluation protocols, while the *iso*PARAMS variant outperforms the dense baseline on eight of nine tasks, consistent with the validation-loss advantage in Table 1.

Early Exit Analysis As shown in Fig. 3a, GATED RECURRENT TRANSFORMER exhibits superior early-exit performance as an emergent property: at matched inference FLOPs GATED RECURRENT TRANSFORMER exiting with fewer recurrences has better loss than a dense model exiting at an earlier layer. Fig. 3b extends the analysis across model scales. Because uniform depth sampling (Section 3.4) trains all intermediate recurrent states to predict final losses, the model provides a continuous “compute–quality dial”. This allows a single checkpoint to transition from fast, shallow inference to high-quality deep inference without re-training or auxiliary losses.

In Figure 4, we provide a qualitative analysis of how next-token predictions evolve across recurrence steps. Our findings reveal that GATED RECURRENT TRANSFORMER adapts its computational depth to the semantic demands of the prompt: (1) **Knowledge-intensive** tokens often stabilize early in the recurrence trajectory; (2) **Reasoning-based** tasks demonstrate progressive error correction and sharpening of the probability mass over deeper iterations; and (3) **Open-ended or creative** prompts exhibit continuous linguistic refinement, where deeper recurrence enhances local coherence despite the lack of a singular ground truth. This suggests that the model implicitly utilizes the recurrent core to modulate its internal “thinking time” based on task complexity.

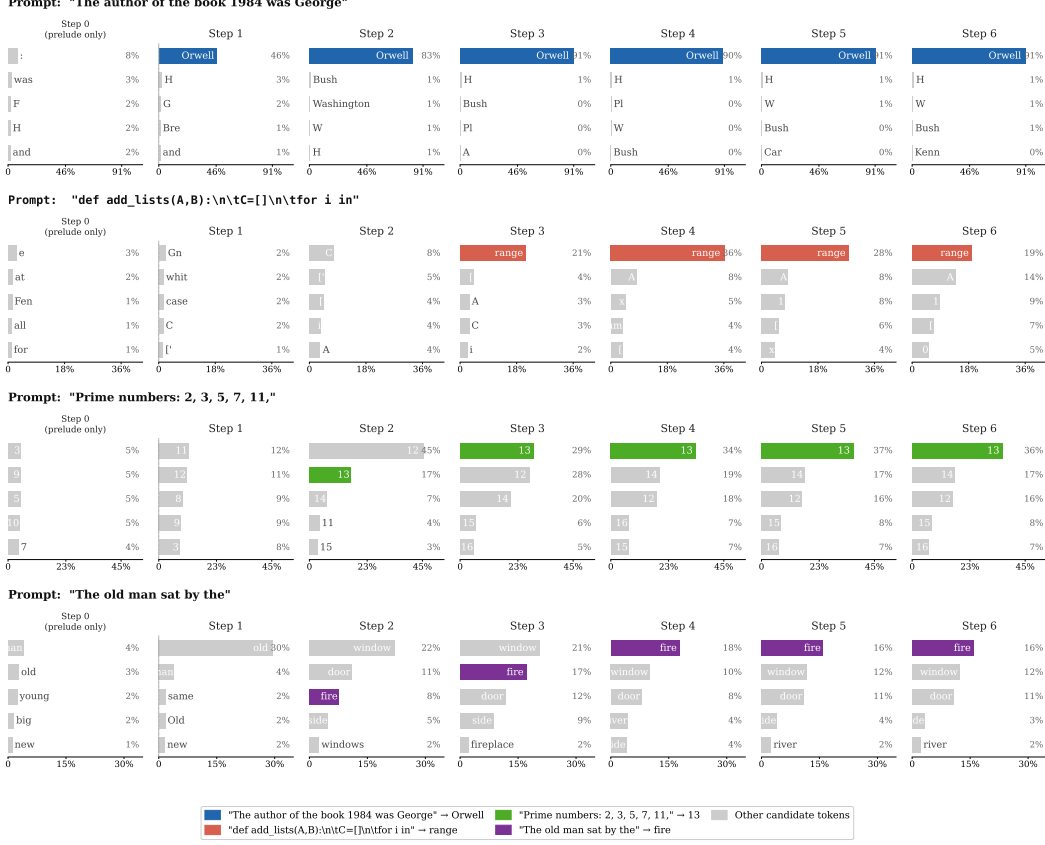


Figure 4: **Next-token probability at each recurrence step.** Each row shows one prompt; the correct answer is highlighted in colour. Prelude-only output (leftmost column) captures partial signal; by step 2 the correct token dominates in three of four prompts. Step labels use 1-indexed notation consistent with the analysis section.

KV Cache Sharing Naively, GATED RECURRENT TRANSFORMER requires R separate KV caches during decoding—one per recurrence step—multiplying memory by R over a standard transformer. Following [10], we evaluate three compressed strategies that each reduce recurrent KV memory over R layers, equivalent to $3.27\times$ reduction across the model: reusing only the *last* step’s cache, only the *first* step’s, or an *averaged* K/V across steps. As shown in Figure 7, we observe that *averaged* leads to best accuracy, while *last* leads to the worst accuracy.

Wall-Clock Latency and Decoding Memory GATED RECURRENT TRANSFORMER executes the same number of block applications as the dense model it is *isoFLOPs*-matched to, distributed over fewer unique weight tensors. Table 3 reports measured generation latency and peak GPU memory on identical hardware (batch size 4, prompt length 1024, 128 generated tokens). Under `torch.compile`, GATED RECURRENT TRANSFORMER-Large costs +10% generation latency for 62% fewer parameters and 59% less peak memory; at medium scale the overhead is +11%. In eager mode both overheads are +23%, so roughly half the gap is kernel-launch overhead from the additional elementwise operations rather than arithmetic.

The $R\times$ -expanded KV cache is a real cost, and whether it erodes the parameter saving depends on batch size. Table 4 reports end-to-end decoding memory (weights + KV, bf16, $T = 1024$) for the medium *isoFLOPs* checkpoint. At $B=1$ decoding is weight-dominated and the parameter reduction lands at $0.55\times$ dense memory even with the naive $R\times$ cache; at $B=32$ the cache dominates and the naive strategy recovers only $0.91\times$. Averaging K/V across recurrence steps brings this to $0.39\times$ while *improving* HellaSwag accuracy over the full cache (33.90 vs. 33.65), which suggests the averaging acts as a mild regulariser at this scale.

Table 3: **Generation latency and peak memory (*iso*FLOPs configurations).** Batch size 4, prompt length 1024, 128 generated tokens, same hardware. Both arms are measured under `torch.compile` and in eager mode.

Model	Config	Params	Eager (ms/tok)	Compiled (ms/tok)	Peak mem.
GPT-2 Medium	24L	354M	2.95	2.54	747 MB
GRT Medium	2+5×4+2	127M	3.62	2.81	402 MB
GPT-2 Large	36L	774M	4.33	3.33	1570 MB
GRT Large	1+5×6+5	293M	5.33	3.67	639 MB

Table 4: **End-to-end decoding memory (medium *iso*FLOPs checkpoint, bf16, $T = 1024$).** Weights + KV cache. Parenthesised values are relative to GPT-2 Medium.

Configuration	HellaSwag	Mem. @ $B=1$	Mem. @ $B=32$
GPT-2 Medium (24L, 354M)	34.45	0.75 GiB (1.00×)	3.66 GiB (1.00×)
GRT <i>iso</i> FLOPs, full $R \times$ KV	33.65	0.41 GiB (0.55×)	3.32 GiB (0.91×)
GRT <i>iso</i> FLOPs, averaged KV	33.90	0.35 GiB (0.47×)	1.44 GiB (0.39×)

Mechanistic Analysis Summary Detailed mechanistic analysis of the large 1+5×6+5 checkpoint is provided in Appendix E; we summarise the key findings here. We observe in Figure 10 that gate, $g^{(r)}$, behavior evolves across recurrence steps: early recurrences ($r \leq 2$) are *write-heavy* (gate values near 0, block output dominates), while later steps transition to *copy-heavy* behavior (gate near 1, residual stream preserved), consistent with a coarse-to-fine computation strategy. Centered Kernel Alignment [42] analysis in Figure 12 confirms that representations across recurrence steps are more similar to one another than representations across layers in a standard transformer of matched depth, suggesting the shared block learns a broadly applicable transformation rather than depth-specialized ones. In Figure 13, we observe that tokens that are more difficult for the model accumulate $10\times$ more improvement across recurrence steps, demonstrating implicit compute routing toward harder inputs without any explicit difficulty signal.

5 Ablations

We study two complementary aspects of GATED RECURRENT TRANSFORMER: the contribution of each architectural component (Section 5) and the model’s behaviour as the training data budget varies (Section 5). All ablations use validation loss as the primary metric.

Component Ablation Figure 5 traces a sequential ablation on the small model at the full 20,000-step budget. Recurrence alone raises validation loss by 0.107 nats: without a mechanism to differentiate steps, the shared block conflates early and late processing into a single weight tensor. Adding prelude and coda boundaries (row 3) recovers 0.035 nats; state noise (row 4) contributes 0.018 more, which also isolates the value of injecting noise at every step rather than only at initialisation. Prelude re-injection into each recurrence (row 5) cuts loss by a further 0.022 nats. The elementwise gate (row 6) is the single largest contributor at **−0.048 nats**.

Data Efficiency As illustrated in Figure 6, we analyze the performance of recurrent depth across varying training data budgets. At smaller model scales, GATED RECURRENT TRANSFORMER consistently maintains a superior Pareto frontier compared to the dense baseline across all observed data volumes. At larger scales, a more complex “crossover” dynamic emerges: while the dense model shows an initial lead under low-data regimes, the gap narrows significantly as the token budget increases. For the Large-scale configuration, the trajectories converge at higher data budgets, suggesting that recurrent sharing is particularly effective at saturating parameter capacity when provided with sufficient data.

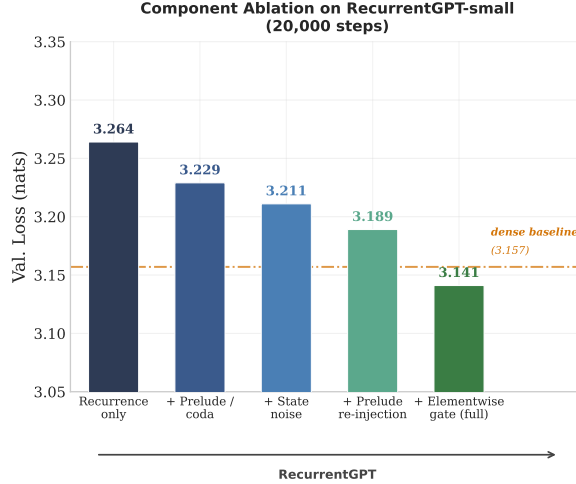


Figure 5: **Component ablation on GATED RECURRENT TRANSFORMER-small (20,000 steps).** Each bar adds one component to the one on its left. The dashed line is the dense 12-layer baseline at 3.157, trained identically; only the full model falls below it. The 2,000-step counterpart on the medium configuration is in Appendix B.2.

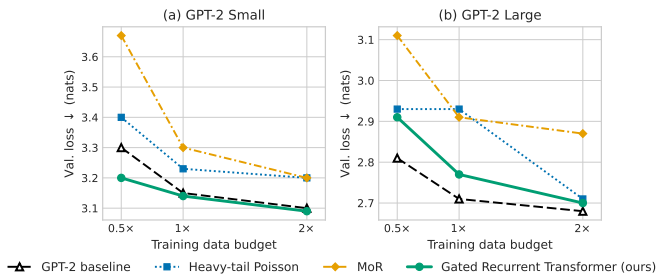


Figure 6: **Validation loss vs. training data budget.** Validation loss vs. training data budget at small and large scale.

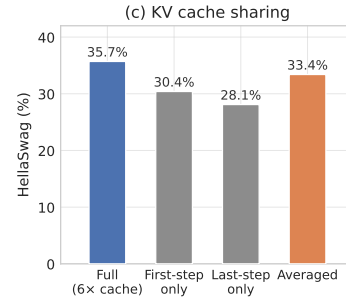


Figure 7: **KV cache sharing.** HellaSwag accuracy (%) for the large *iso*FLOPS checkpoint.

6 Conclusion

GATED RECURRENT TRANSFORMER demonstrates that a learned per-element gate conditioning on the current hidden state, the fixed prelude output, and injected stochastic noise is a principled strategy for decoupling model depth from parameter count. Under an *iso*FLOPS constraint, GATED RECURRENT TRANSFORMER stores only 36–37% of the dense baseline’s unique parameters while beating both MoR [25] and heavy-tail Poisson sampling at every scale; under an *iso*PARAMS constraint, deeper recurrence yields consistent validation-loss gains (Table 1) and downstream improvements of up to +8.29 points (Table 2). Three limitations bound the current work: the recurrence depth R is fixed at inference with no per-token halting [23]; the gate bias and noise magnitudes may require re-tuning beyond the GPT-2 family; and the scale-dependent optimal sharing fraction warrants systematic study. Future directions include dynamic per-token halting and knowledge distillation from a dense language model into a recurrent student.

References

- [1] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.
- [2] Alan Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1936. doi: 10.2307/2268810.
- [3] Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=4FWAwZtd2n>.
- [4] Learning to Reason with LLMs. URL <https://openai.com/index/learning-to-reason-with-llms/>.
- [5] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- [6] Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: A theoretical perspective. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=qHrADgAdYu>.
- [7] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations*, 2020.
- [8] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HyzdRiR9Y7>.

- [9] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach, 2025. URL <https://arxiv.org/abs/2502.05171>.
- [10] Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenchua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling latent reasoning via looped language models, 2025. URL <https://arxiv.org/abs/2510.25741>.
- [11] Ahmadreza Jeddi, Marco Ciccone, and Babak Taati. Loopformer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=RzYXb5YWBs>.
- [12] Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise loRA. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=WwpYS0kkCt>.
- [13] Daniel Bolya et al. Depth as modulation in weight-sharing transformers. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=wm9jRInse3>.
- [14] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986. doi: 10.1038/323533a0.
- [15] P.J. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990. doi: 10.1109/5.58337.
- [16] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. doi: https://doi.org/10.1207/s15516709cog1402_1. URL https://onlinelibrary.wiley.com/doi/abs/10.1207/s15516709cog1402_1.
- [17] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [18] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar, October 2014. Association for Computational Linguistics. doi: 10.3115/v1/D14-1179. URL <https://aclanthology.org/D14-1179/>.
- [19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423/>.
- [20] Yeskendir Koishikenov, Aldo Lipani, and Nicola Cancedda. Encode, think, decode: Scaling test-time reasoning with recursive latent thoughts, 2025. URL <https://arxiv.org/abs/2510.07358>.
- [21] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5484–5495, Online and Punta Cana, Dominican Republic, November 2021.

- Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.446. URL <https://aclanthology.org/2021.emnlp-main.446/>.
- [22] Ruike Zhu, Hanwen Zhang, Kevin Li, Tianyu Shi, Yiqun Duan, Chi Wang, Tianyi Zhou, Arindam Banerjee, and Zengyi Qin. Beyond parameters: Exploring virtual logic depth for scaling laws, 2025. URL <https://arxiv.org/abs/2506.18233>.
 - [23] Alex Graves. Adaptive computation time for recurrent neural networks, 2017. URL <https://arxiv.org/abs/1603.08983>.
 - [24] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024. URL <https://arxiv.org/abs/2404.02258>.
 - [25] Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=QuqsEIVWIG>.
 - [26] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, 2019.
 - [27] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022. URL <https://arxiv.org/abs/2203.15556>.
 - [28] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling, 2023. URL <https://arxiv.org/abs/2304.01373>.
 - [29] Gilad Yehudai, Clayton Sanford, Maya Bechler-Speicher, Orr Fischer, Ran Gilad-Bachrach, and Amir Globerson. Depth-width tradeoffs in algorithmic reasoning of graph tasks with transformers, 2026. URL <https://arxiv.org/abs/2503.01805>.
 - [30] Sean Michael McLeish, John Kirchenbauer, David Yu Miller, Siddharth Singh, Abhinav Bhatele, Micah Goldblum, Ashwinee Panda, and Tom Goldstein. Gemstones: A model suite for multi-faceted scaling laws. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=iZk78dZ1Ap>.
 - [31] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL <https://arxiv.org/abs/1607.06450>.
 - [32] Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with LSTM. *Neural Computation*, 12(10):2451–2471, 2000.
 - [33] Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. An empirical exploration of recurrent network architectures. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 2342–2350, 2015.
 - [34] Google DeepMind. Gemma 3n model overview. <https://ai.google.dev/gemma/docs/gemma-3n>, May 2025. Introduced Per-Layer Embeddings (PLE) for on-device LLMs.
 - [35] Google DeepMind. Gemma 4 model overview. <https://ai.google.dev/gemma/docs/core>, April 2026. Accessed: 2026-05-06. Last updated 2026-04-16.
 - [36] Yanxi Chen, Xuchen Pan, Yaliang Li, Bolin Ding, and Jingren Zhou. Ee-llm: Large-scale training and inference of early-exit large language models with 3d parallelism. In *The Forty-first International Conference on Machine Learning*, 2024.

- [37] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: Enabling early exit inference and self-speculative decoding. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12622–12642, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.681. URL <https://aclanthology.org/2024.acl-long.681/>.
- [38] Andrej Karpathy. NanoGPT. <https://github.com/karpathy/nanoGPT>, 2022.
- [39] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [40] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [41] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.
- [42] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 3519–3529. PMLR, 2019.

A Implementation Details

```

def grt_forward(x, model, R, r_min=1):          # R = max recurrence depth
    h = token_embed(x) + pos_embed(x)          # [S, d]
    for block in model.prelude_blocks:
        h = h + block(h)
    h_pre = h                                  # h^(0) = h^(pre): frozen anchor
    r = R if not training else uniform_sample(r_min, R) # depth sampling
    for step in range(r):                      # r in 0, ..., R-1
        eps = sample_gaussian(0, sigma**2)
        h_tilde = W_proj( cat([h + eps, h_pre]) ) # concat -> project to d
        o = B_shared(h_tilde)                  # same weights every step
        eps_g = sample_gaussian(0, sigma_g**2)
        g = sigmoid(f_gate(LN(h), LN(h_pre))/tau + eps_g) # gate in [0,1]^{Sxd}
        h = g * h + (1 - g) * o                # gated residual update
    for block in model.coda_blocks:
        h = h + block(h)
    return lm_head(h)                          # [S, |V|]

```

Listing 1: **GATED RECURRENT TRANSFORMER forward pass.**

A.1 Hyperparameter Table

Table 5 lists the full training configuration for every run reported in the main paper.

Table 5: **Training configurations.** p : prelude blocks; b : shared blocks; R : recurrence steps; c : coda blocks; d : embedding dimension; h : attention heads. Batch tokens = batch_size $\times$ gradient_accumulation $\times$ sequence_length $\times$ num_GPUs. All runs use AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$), weight decay 0.1, gradient clip 1.0, bfloat16 mixed precision, cosine LR schedule.

Run	p	b	R	c	d	h	Batch tokens	Peak LR	Steps	Hardware
GPT-2 Small (baseline)	—	—	—	—	768	12	491,520	6×10^{-4}	20k	2 $\times$ H200
GRT Small (isoFLOP)	1	1	10	1	768	12	491,520	6×10^{-4}	20k	2 $\times$ H200
GPT-2 Medium (baseline)	—	—	—	—	1024	16	491,520	6×10^{-4}	20k	2 $\times$ H200
GRT Medium (isoFLOP)	2	5	4	2	1024	16	491,520	6×10^{-4}	20k	2 $\times$ H200
GRT Medium (isoParam)	2	20	4	2	1024	16	491,520	6×10^{-4}	20k	2 $\times$ H200
GPT-2 Large (baseline)	—	—	—	—	1280	20	491,520	6×10^{-4}	20k	2 $\times$ H200
GRT Large (isoFLOP)	1	5	6	5	1280	20	491,520	6×10^{-4}	20k	2 $\times$ H200
GRT Large (isoParam)	3	30	6	3	1280	20	491,520	6×10^{-4}	20k	2 $\times$ H200

All recurrent runs use gate bias initialisation +4 ($g \approx 0.98$ at init), state noise $\sigma_x = 0.1$, gate noise $\sigma_g = 0.1$, and gate temperature $\tau = 1.0$. During training, recurrence depth is sampled uniformly from $\{1, \dots, R\}$ at each training step.

B Additional Ablations

B.1 Run-to-Run Variance Across Seeds

Table 6 reports three independent training runs (seeds 1337, 42, 123) of the small *isoFLOP*s configuration and of its dense counterpart, all at the full 20,000-step budget. GATED RECURRENT TRANSFORMER averages 3.145 ± 0.004 against 3.188 ± 0.056 for the dense baseline. GATED RECURRENT TRANSFORMER’s worst seed (3.148) still falls below the dense baseline’s best (3.154). Seed 1337 is the run reported throughout the main paper.

B.2 Component Ablation at Short Horizon

Table 7 reports the sequential ablation of Section 5 run on the *medium* configuration at 2,000 steps. At this horizon the ordering of the last two rows is inverted relative to the full 20,000-step run:

Table 6: **Three-seed validation loss on the small configuration** (20,000 steps). Both models use the *iso*FLOPS setting of Table 1. Lower is better.

Model	Seed 1337	Seed 42	Seed 123	Mean $\pm$ std
Dense baseline (12L)	3.154	3.156	3.253	3.188 ± 0.056
GRT (1+1 $\times$ 10+1)	3.141	3.146	3.148	3.145 ± 0.004

prelude re-injection contributes -0.198 nats against the gate’s -0.115 . We report both because the comparison is itself informative — the structural components land early, whereas the gate is a learned mechanism whose contribution accrues over training.

Table 7: **Component ablation on GATED RECURRENT TRANSFORMER-medium at 2 000 steps.** Short-horizon counterpart to Figure 5. The dense baseline (row 1) is a standard 24-layer GPT trained identically.

	Configuration	Val. loss	Δ
1	Dense baseline (non-recurrent)	3.621	—
2	Recurrence	4.163	+0.542
3	+ Prelude / coda (2+ ₋ + 2)	4.118	-0.045
4	+ State noise ($\sigma_x=0.1$)	4.099	-0.019
5	+ Prelude re-injection	3.901	-0.198
6	+ Elementwise gate (full GATED RECURRENT TRANSFORMER)	3.786	-0.115

B.3 Design Comparison of Recurrent-Depth Methods

Table 8 sets the small-scale results of Table 1 beside the design choices that produce them. MoR and Ouro gate the update, but Ouro supervises every iteration and MoR needs a router; heavy-tail Poisson shares strictly and updates unconditionally; RRT is input-independent and depth-fixed. GATED RECURRENT TRANSFORMER is the only column combining full sharing, an input-dependent update, and variable inference depth.

Table 8: **Design properties of recurrent-depth methods**, with small-scale *iso*FLOPS results for reference. Training times are wall-clock for the small configuration at 20,000 steps on identical hardware, and are *not* comparable against the dense column: the dense baseline was trained with `torch.compile` enabled, whereas every recurrent method including our own ran with compilation disabled, which did not work on our hardware. The dense column is therefore optimistic; the recurrent columns remain comparable to one another.

Property	Dense	MoR [25]	Poisson [9]	RRT [12]	Ouro [10]	GRT (ours)
Full weight sharing	n/a	Yes	Yes	No (LoRA)	Yes	Yes
Variable inference depth	No	Yes	Yes	No	Yes	Yes
Input-dependent gating	No	Yes	No	No	Yes	Yes
Per-step noise injection	No	No	Init only	No	No	Yes
Per-iteration loss	No	No	No	No	Yes	No
Emergent early exit	No	Yes	Yes	No	Yes	Yes
Training time (small)	3h18m	12h19m	4h16m	6h42m	8h20m	6h18m
Small val. loss $\downarrow$	3.15	3.30	3.23	3.14	3.19	3.14

B.4 Gate Temperature Sensitivity

The gate temperature τ (Eq. 5) controls the sharpness of the sigmoid. We sweep $\tau \in \{0.5, 1.0, 2.0\}$ on the medium configuration at 5,000 steps. Validation loss is 3.62, 3.60, and 3.63 respectively, indicating that the optimal gate temperature is $\tau = 1.0$.

B.5 Noise Magnitude Sensitivity

We ablate state noise $\sigma_x \in \{0.0, 0.05, 0.1, 0.2\}$ on the medium configuration at 5,000 steps. Removing noise entirely ($\sigma_x = 0.0$) increases validation loss by 0.031 nats relative to $\sigma_x = 0.1$; larger noise ($\sigma_x = 0.2$) degrades by 0.018 nats. The optimal range is $\sigma_x \in [0.05, 0.1]$; we use 0.1 throughout.

B.6 Gate Bias and State Noise at Full Training Horizon

The sweeps above are at 5,000 steps. We repeated both at the full 20,000-step budget on the small configuration to check that the response surface does not sharpen at convergence. It does not. Moving the gate bias from the default +4 down to 0 costs 0.019 nats, and -2 recovers to 3.151; the surface is shallow and non-monotonic rather than peaked, and no setting produced a loss spike or divergence. State noise behaves the same way: removing it costs 0.018 nats and doubling it costs 0.019, a symmetric optimum rather than a value that has to be hit precisely. This is weaker evidence than a cross-architecture study, but it does suggest the mechanism does not depend on the exact bias value.

Table 9: Gate-bias sensitivity, small config at 20,000 steps.

Gate bias	Val. loss ↓
+4 (default)	3.141
+2	3.152
0	3.160
-2	3.151

Table 10: State-noise sensitivity, small config at 20,000 steps.

σ_x	Val. loss ↓	Δ
0	3.229	+0.018
0.1 (default)	3.211	—
0.2	3.230	+0.019

C Additional Qualitative Results

C.1 Per-Token Gate Activations Across Recurrence Steps

Figure 8 shows $\bar{g}_t^{(r)} = \text{mean}_d(g_{t,:}^{(r)})$ — the elementwise gate averaged over the model dimension — for each token position t and recurrence step r , on four example prompts. Brighter (yellow-green) cells indicate write-heavy positions where the gate is open ($\bar{g} \approx 0.70$ – 0.82) and the shared block’s proposal is substantially absorbed; darker (purple) cells indicate copy-heavy positions ($\bar{g} \approx 0.95$ – 1.00) where the hidden state passes through largely unchanged.

Three consistent observations emerge across all four prompts. First, **the per-step mean gate rises monotonically from step 2 onward**: means fall from ~ 0.87 – 0.89 at step 1 to a minimum around step 2 (write-heavy phase), then recover steadily toward ~ 0.88 – 0.91 at step 6 (copy-heavy phase). Second, **write-heavy positions are structurally consistent across steps**: the same columns that light up at step 1 tend to be the same at step 6, indicating that token identity — not recurrence depth — drives the gate primary signal, with the gate progressively dimming those writes over steps rather than shifting which tokens are written. Third, **content words and syntactically load-bearing tokens receive lower gate values than function words**: in “The old man sat by the”, *man* and *sat* are visibly brighter than *the* and *by*; in the code prompt, the operator and identifier tokens show write-heavy behaviour while keywords like `def` and `for` are copy-heavy. This implicit routing — without any explicit per-token mechanism — arises purely from the gate conditioning on both the evolving hidden state $x^{(r)}$ and the fixed prelude anchor h .

D Broader Impact

Parameter-efficient language models have direct environmental and accessibility benefits: a GATED RECURRENT TRANSFORMER checkpoint at 37% of a baseline’s parameter count requires proportionally less GPU memory at serving time, enabling deployment on hardware that cannot accommodate the full dense model and reducing energy consumption per inference step. Recurrent depth also provides a natural compute-quality tradeoff at inference time—the early-exit capability of Section 4 allows a single trained model to serve requests at multiple quality-FLOPs operating points without retraining.

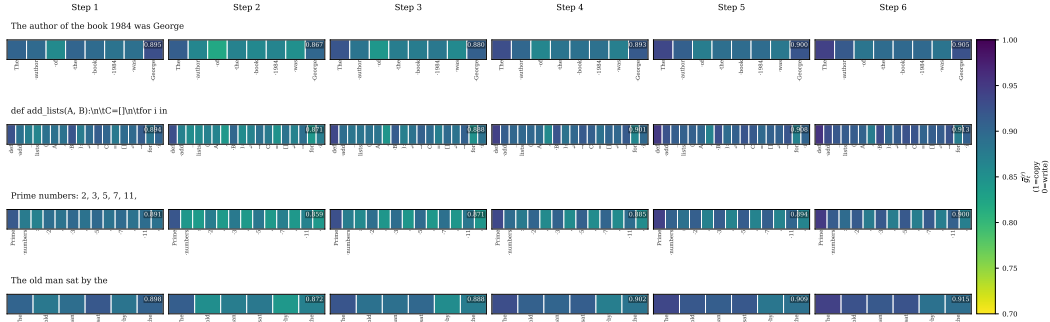


Figure 8: **Per-token gate activation $\bar{g}_t^{(r)}$ across recurrence steps.** Each row of cells corresponds to one recurrence step (columns = token positions). Colour encodes $\bar{g}_t^{(r)} \in [0.70, 1.00]$: bright yellow-green = write-heavy (gate open, block output substantially absorbed); dark purple = copy-heavy (hidden state passed through unchanged). Token labels are shown below each step column. The per-step mean (annotated top-right of each panel) rises from step 2 onward, reflecting the write-to-copy transition identified in Figure 10(b). Structurally informative tokens (content words, operators) consistently receive lower gate values than function words and punctuation across all steps.

The risks associated with GATED RECURRENT TRANSFORMER are not specific to its architecture: as a capable language model trained on web text, it inherits the standard dual-use risks of any such system, including the potential to generate misinformation or harmful content. These risks are addressed by standard deployment safeguards (content filtering, output monitoring, and responsible release practices) that apply equally to all current language models; the recurrent architecture does not introduce qualitatively new threat vectors.

E Recurrence Dynamics Analysis

Beyond aggregate validation loss, we probe *what* the shared block computes at each recurrence step $r \in \{1, \dots, 6\}$ using the large $1+5 \times 6+5$ checkpoint. Diagnostics are run on a validation split with inference noise disabled. We report ten interlocking findings—grouped first into behavioural diagnostics (Section E.1–Section E.6) and then into mechanistic analyses of weights and attribution (Section E.7–Section E.10)—before drawing them together in Section E.11.

E.1 Rapid Convergence of Loss Across Recurrence Steps

Figure 9(a) traces the per-step validation loss as the shared block is applied iteratively. The prelude output alone yields a loss of 5.29; after a single application of the shared core the loss drops to 3.77—a reduction of 1.52 nats (natural-log-scale bits; 1 nat = $\log_2 e \approx 1.44$ bits) in one step. By step 2 the loss reaches 3.14, and the remaining four steps account for only an additional 0.46 nats, converging to the final value of 2.68. Stated differently, the first two recurrence steps account for roughly 77% of the total loss reduction from the prelude representation to the final output.

This front-loaded profile is confirmed by the KL-divergence trajectory in Figure 9(b), which measures residual uncertainty relative to the final-step output distribution. The KL falls from 2.61 at the prelude to 0.46 by step 2 and reaches 0.014 at step 5, indicating that the output distribution is essentially committed after the fourth step. As a lower bound on the gate’s utility, forcing the gate to zero throughout (the “no-recurrence” ablation) yields a catastrophic loss of 12.03, confirming that the iterative refinements are not redundant. The complementary ablation—forcing the gate to one throughout, so that every block proposal is discarded and the hidden state is never updated—yields a loss of 5.26, equivalent to the prelude-only output (5.26 at step $r = 0$). This confirms that recurrence with a fully open gate is vacuous: without selective writing, six recurrence steps reduce to a single prelude pass. Together, Gate=0 (12.03) and Gate=1 (5.26) bracket the trained model (2.68): the learned gate neither skips recurrence nor blindly overwrites state, but acquires a fine-grained elementwise balance between retention and update.

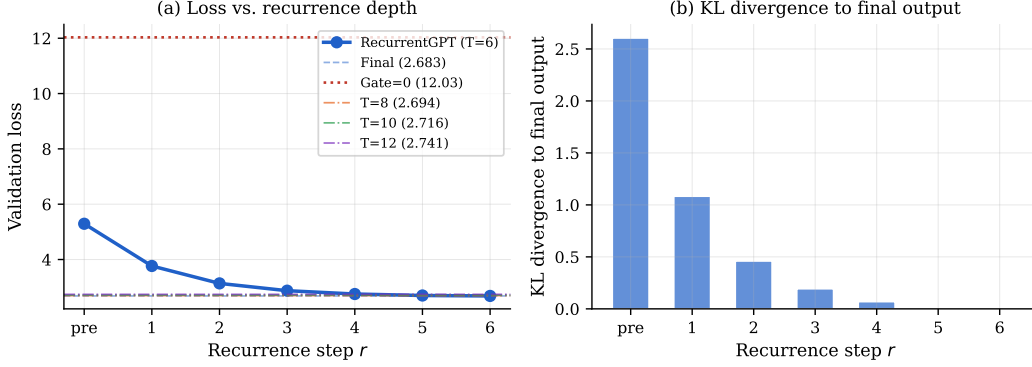


Figure 9: **Loss convergence across recurrence steps.** (a) Validation loss at each recurrence step r for the trained $1+5 \times 6+5$ model. The “Gate=1” baseline (loss 5.26, green dashed) equals the prelude-only output, confirming that selective writing is essential: a fully open gate discards every block proposal, leaving the hidden state unchanged across all six steps. The “Gate=0” baseline (loss 12.03) confirms that recurrence is essential, not a skip connection. Extended inference at $R \in \{8, 10, 12\}$ does not improve beyond $R = 6$ (2.69, 2.72, and 2.74 respectively), indicating representational convergence. (b) KL divergence of each step’s output distribution to the final-step distribution; 95% of the residual uncertainty is resolved within four recurrence steps.

We also evaluate extending the recurrence depth at inference time beyond the $R = 6$ steps seen during training. Applying the trained shared block for $R \in \{8, 10, 12\}$ steps marginally *degrades* performance (2.69, 2.72, 2.74 respectively), suggesting the model has converged representationally by step 6 and that additional steps introduce noise rather than refinement.

Takeaway

77% of the loss reduction from prelude to final output occurs within the first two recurrence steps; the remaining four steps provide targeted corrections. The model’s output distribution is essentially committed by step 4.

E.2 Gate Behaviour: Write-Heavy Early, Copy-Heavy Late

The elementwise gate $\mathbf{g}^{(r)} \in [0, 1]^{S \times d}$ controls how much of the shared block’s proposal is written into the residual stream at each step. Figure 10(a) plots the per-step mean and standard deviation. The gate is most open—and most variable—in the middle steps (steps 2–3, mean ≈ 0.82 , std ≈ 0.18), corresponding to the highest information-writing regime, and tightens progressively toward step 6 (mean 0.87, std 0.12).

Figure 10(b) quantifies this transition via the fraction of dimensions where the gate is near-saturated. Copy-saturated dimensions ($g > 0.95$) grow monotonically from 19.7% at step 1 to 28.8% at step 6, while write-saturated dimensions ($g < 0.05$) remain negligibly rare ($< 10^{-4}$) throughout. This asymmetry—abundant copying, scarce full-overwriting—indicates the model prefers selective blending over clean state replacement.

The effective gate openness (Figure 10(c)), measured as the ratio

$$\rho^{(r)} = \frac{\|\mathbf{h}^{(r)} - \mathbf{h}^{(r-1)}\|_2}{\|\mathbf{o}^{(r)} - \mathbf{h}^{(r-1)}\|_2} = \frac{\|(1 - \mathbf{g}^{(r)}) \odot (\mathbf{o}^{(r)} - \mathbf{h}^{(r-1)})\|_2}{\|\mathbf{o}^{(r)} - \mathbf{h}^{(r-1)}\|_2}, \quad (6)$$

where the numerator is the norm of the *applied update* (actual change to the hidden state) and the denominator is the norm of the *proposal* (the full update the block would write if the gate were fully open), decreases monotonically from 0.182 at step 1 to 0.066 at step 6. Taken together, the gate transitions the model from a *write-heavy* regime in early steps—where large fractions of the proposal are incorporated to rapidly restructure the representation—to a *copy-heavy* regime in later steps, where the representation is largely preserved and only small targeted corrections are applied.

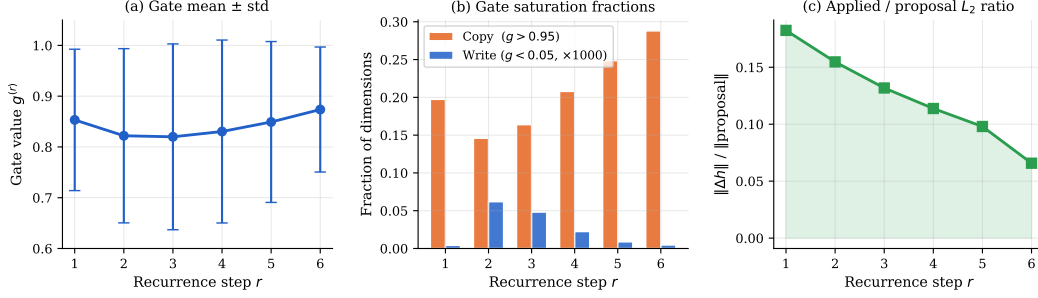


Figure 10: **Gate statistics across recurrence steps.** (a) Gate mean and standard deviation. (b) Fraction of copy-saturated ($g > 0.95$) and write-saturated ($g < 0.05$, scaled $\times 1000$) dimensions per step. Copy-saturation grows steadily while write-saturation stays negligible. (c) Effective gate openness (applied/proposal L_2 ratio), declining monotonically—early steps write substantially, late steps refine conservatively.

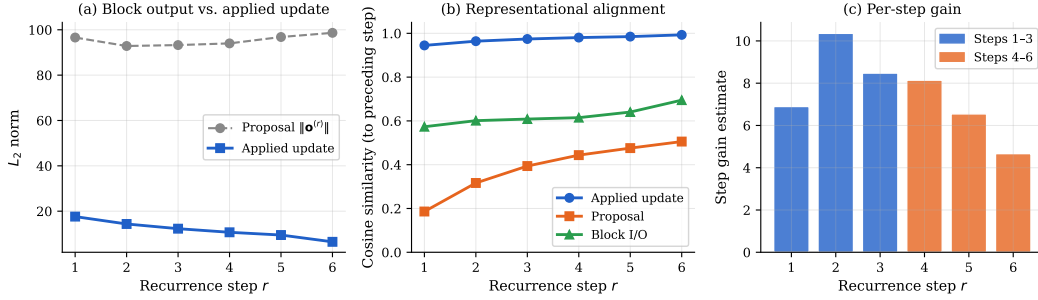


Figure 11: **Update dynamics across recurrence steps.** (a) L_2 norms of the block’s raw proposal (near-constant, ≈ 94) and the gate-applied update (declining $17.6 \rightarrow 6.5$); the divergence is caused entirely by gate closure. (b) Cosine similarity between successive representations. (c) Local step gain estimate, peaking at step 2 and declining thereafter.

Takeaway

The gate transitions from a write-heavy regime (steps 1–2, mean ≈ 0.82) to a copy-heavy regime (steps 3–6, mean ≈ 0.87), with effective gate openness declining monotonically from 0.18 to 0.07.

E.3 Update Dynamics: Diminishing Magnitude, Increasing Alignment

Figure 11(a) separates two quantities at each step: the L_2 norm of the shared block’s raw proposal $\mathbf{o}^{(r)}$, and the L_2 norm of the gate-applied update that actually enters the residual stream. The proposal norms are nearly constant across all six steps (93–99), possibly indicating that the shared block does not itself “know” it is being applied repeatedly—its output magnitude is stationary. The applied update, by contrast, declines from 17.6 at step 1 to 6.5 at step 6, driven entirely by the gate becoming more closed.

Figure 11(b) shows cosine similarity between successive representations, $\cos_{\text{sim}}(\mathbf{h}^r, \mathbf{h}^{r-1}) = \frac{\mathbf{h}^r \cdot \mathbf{h}^{r-1}}{\|\mathbf{h}^r\| \|\mathbf{h}^{r-1}\|}$. The proposal cosine similarity grows from 0.19 at step 1 to 0.51 at step 6, indicating that the block and the hidden state increasingly agree on direction as the representation stabilises. The applied update cosine similarity is already high at step 1 (0.94) and approaches 0.99 by step 6, confirming that late-step corrections are directionally consistent refinements.

The per-step gain estimate (Figure 11(c)) peaks at step 2 (10.4), declining to 4.7 at step 6. This concave gain profile, in which the most productive computation occurs before representational alignment is achieved, is consistent with the loss convergence plots in Section E.1.

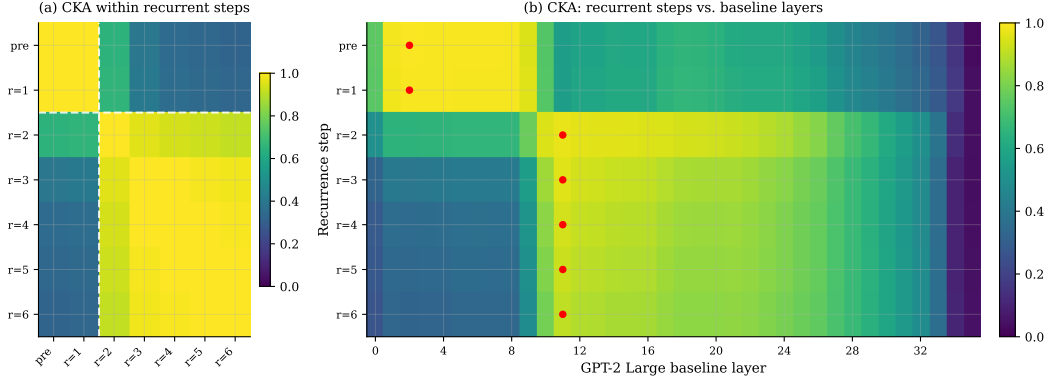


Figure 12: **CKA representational geometry.** (a) Within-model CKA; the sharp discontinuity between steps 1–2 and steps 3–6 (dashed lines) reveals a two-phase structure. (b) Cross-model CKA against GPT-2 Large; early steps map to shallow baseline layers, all later steps peak at layer ≈ 11 .

E.4 Representational Geometry: Two-Phase Structure

We use Centered Kernel Alignment [CKA; 42] to compare hidden-state representations across recurrence steps and against the corresponding layer representations of the GPT-2 Large dense baseline.

Within-model CKA (Figure 12(a)). The 7×7 CKA matrix reveals a two-phase structure. Representations at step 0 (prelude) and step 1 are nearly identical ($\text{CKA} = 0.996$), while both are sharply dissimilar from all subsequent steps ($\text{CKA} \approx 0.33\text{--}0.65$ with steps ≥ 2). Steps 2 through 6 form a tightly cohesive cluster (pairwise $\text{CKA} \geq 0.907$), within which similarity decreases only gradually with step distance. This suggests the model undergoes a *representational phase transition* between steps 1 and 2.

Cross-model CKA (Figure 12(b)). Prelude and step 1 representations align maximally with the shallow layers of the baseline (peak at layer 2, $\text{CKA} = 0.993$), while steps 2 through 6 all peak at baseline layer 11 ($\text{CKA} = 0.97\text{--}0.90$ declining). No recurrent step aligns strongly with the deep layers of the baseline (layers > 20 , $\text{CKA} < 0.50$), confirming that recurrent depth compresses the computational path but does not replicate the later representational hierarchy of the dense model.

Takeaway

A representational phase transition occurs between steps 1 and 2: early steps resemble shallow baseline layers, while steps 2–6 all align with mid-depth baseline layer 11, compressing the computational path without replicating the deep-layer hierarchy.

E.5 Differential Routing by Token Difficulty

To test whether the gate allocates computation to where it is most needed, we sort tokens by their loss after the prelude (step 0) into ten deciles and track the total loss improvement accumulated over six recurrence steps. Figure 13(a) shows a near-linear relationship between initial token difficulty and total improvement ($R^2 = 0.998$): the easiest decile gains only 0.49 nats, while the hardest decile gains 5.02 nats—a ten-fold difference. The gate difficulty correlation—whether tokens with higher loss also receive lower (more open) gate values—is mildly negative across all steps (-0.07 to -0.04), consistent with the gate allowing slightly more writing for uncertain tokens.

E.6 Fixed-Point Convergence and Effective Rank

Figure 14(a) tracks the effective rank of the hidden-state tensor at each step. The rank increases from 0.546 at the prelude to 0.635 by step 3, after which it plateaus, indicating that iterative updates progressively spread information across more hidden dimensions. The effective rank of the δ

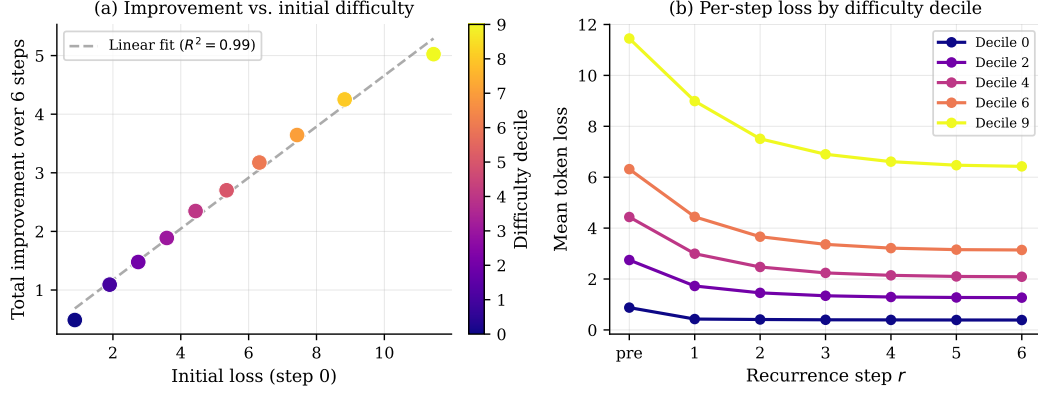


Figure 13: **Token difficulty routing.** (a) Total improvement over six steps vs. initial difficulty decile ($R^2 = 0.998$); recurrence implicitly allocates more work to uncertain tokens. (b) Per-step loss curves for selected deciles.

subspace—the directions along which the hidden state changes in later steps—is 0.69, higher than the rank of the representation itself, suggesting that late-step updates refine a diverse set of features.

Figure 14(b) plots the normalised fixed-point convergence metric, defined as

$$\delta_{\text{fp}}^{(r)} = \frac{\|\mathbf{h}^{(r)} - \mathbf{h}^{(r-1)}\|_F}{\|\mathbf{h}^{(r-1)}\|_F}, \quad (7)$$

the Frobenius-norm magnitude of the update relative to the current representation, which declines monotonically from 0.324 at step 1 to 0.112 at step 6, confirming that the recurrent dynamics are contractive.

Table 11 reports a sanity check via anchor ablations—swapping the fixed prelude output $\mathbf{h}^{(\text{pre})}$ fed at each recurrence step for alternative anchors while keeping the gate and blocks identical. Crucially, the “previous hidden state” row uses $\mathbf{h}^{(r-1)}$ as anchor (i.e. the anchor drifts with the recurrence rather than being held fixed), while the learned gate remains fully active and adaptive. The 0.70-nat gap between this condition (3.38) and the trained model (2.68) therefore quantifies the value of providing a *stable, fixed reference point* at every step, not merely the value of the gate.

Table 11: Anchor ablation: effect of replacing $\mathbf{h}^{(\text{pre})}$ with alternative anchors at each recurrence step. The “prev. hidden state” anchor drifts at each step ($\mathbf{h}^{(r-1)}$) while the learned adaptive gate remains active throughout. The 0.70-nat gap between that condition and the trained model isolates the value of holding the prelude output *fixed* as a stable context reference.

Anchor type	Val. loss (nats)	Δ vs. trained
Zeros	8.08	+5.40
Input embedding	3.73	+1.05
Prev. hidden state $\mathbf{h}^{(r-1)}$	3.38	+0.70
Trained model (fixed $\mathbf{h}^{(\text{pre})}$)	2.68	—

E.7 Gate Attribution: Current State Gradually Supersedes the Anchor

The elementwise gate (Eq. 5) reads from two sources simultaneously: $\hat{\mathbf{x}}^{(r)} \triangleq \text{LN}(\mathbf{h}^{(r-1)})$, the LayerNorm-normalised current hidden state,¹ and $\hat{\mathbf{h}} \triangleq \text{LN}(\mathbf{h}^{(\text{pre})})$, the normalised prelude anchor. The gate MLP $f_{\mathbf{g}}$ takes their concatenation as input; we write its first linear layer weight as $W_{\mathbf{g}} = [W_x \mid W_h] \in \mathbb{R}^{d_{\text{gate}} \times 2d}$, where $W_x \in \mathbb{R}^{d_{\text{gate}} \times d}$ acts on $\hat{\mathbf{x}}^{(r)}$ and $W_h \in \mathbb{R}^{d_{\text{gate}} \times d}$ acts on $\hat{\mathbf{h}}$. To quantify which source drives the gate at each step, we measure how much variance in the actual per-token gate

¹We write $\hat{\mathbf{x}}^{(r)}$ rather than x to avoid confusion with token IDs $(x_1, \dots, x_S)$ introduced in the Preliminaries.

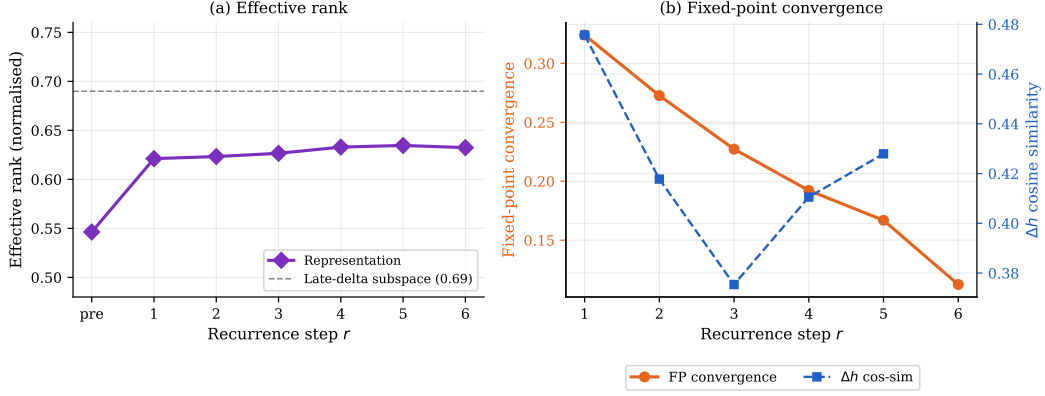


Figure 14: **Effective rank and fixed-point convergence.** (a) Effective rank grows with depth; the late-delta subspace rank (dashed) is higher than the representation itself. (b) Fixed-point convergence (orange) declines monotonically; Δh cosine similarity (blue) increases—both consistent with convergent refinement.

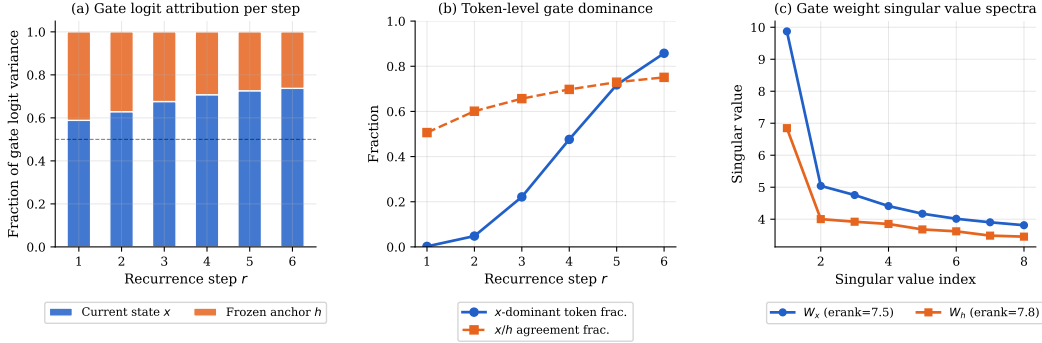


Figure 15: **Gate attribution across recurrence steps.** (a) Fraction of gate logit variance from current state x (blue) vs. frozen anchor h (orange); the crossover from rough parity at step 1 to 74%/26% at step 6 is explained by growing divergence between $x^{(r)}$ and h . (b) Token-level x -dominance and x/h sign-agreement fractions. (c) Gate weight singular value spectra; the 44% gap in leading singular values (9.87 vs 6.85) is the structural origin of the state’s eventual dominance.

logit each half explains. As seen in Figure 15(a), at step 1 the split is nearly even: 59% current state, 41% anchor. By step 6 this shifts substantially—74% current state, 26% anchor.

Figure 15(b) shows the same trend at the token level: the fraction of positions where the state half produces a larger absolute gate logit than the anchor grows from a near-zero 0.2% at step 1 to 86% at step 6. The answer to *why* is present in the weights before any forward pass: we find in Figure 15(c) that the leading singular value of W_x is 9.87 versus 6.85 for W_h , giving the state half structurally greater sensitivity. At step 1 the hidden state is close to the prelude output, so the gap is small; as $x^{(r)}$ diverges from h across steps, this structural asymmetry is amplified into the monotone attribution shift in Figure 15(a).

Takeaway

By step 6, the gate reads 74% of its signal from the current hidden state and only 26% from the fixed prelude anchor. This shift is structurally predetermined: W_x has a 44% larger leading singular value than W_h , priming the gate to amplify state-driven signals as $\mathbf{h}^{(r)}$ departs from $\mathbf{h}^{(\text{pre})}$.

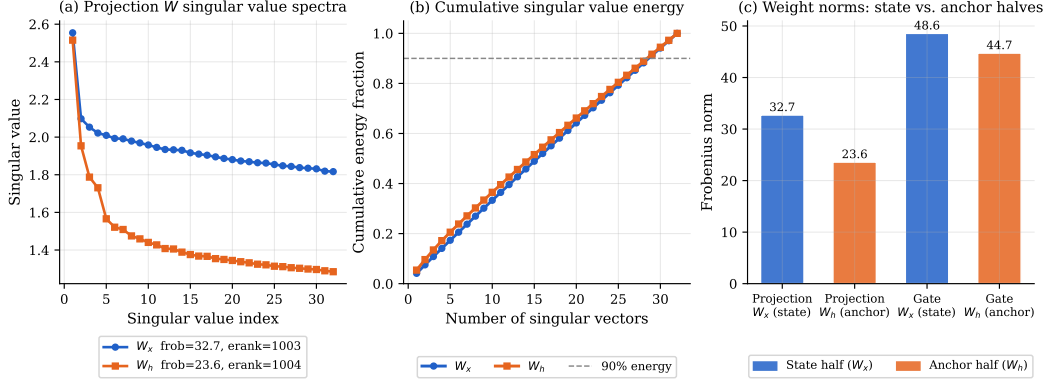


Figure 16: **Recurrent projection weight analysis.** (a) Singular value spectra of W_x and W_h ; both are nearly full-rank. (b) Cumulative energy; both halves reach 90% at similar dimensionalities. (c) Frobenius norms of all four weight halves; the state half (W_x) consistently outweighs the anchor half (W_h) in both modules.

E.8 Projection Weight Analysis: State and Anchor as Complements

Before the shared blocks run, the recurrent projection, W_{proj} , maps the concatenated $[x^{(r)}, h]$ into the d -dimensional input the transformer sees. A natural question is whether this projection simply averages the two sources or treats them as carrying distinct information. The answer is unambiguous: the mean row-wise cosine similarity between the W_x and W_h halves is -0.189 , with only 0.16% of the 1024 rows aligned above a cosine of 0.5.

Both halves are nearly full-rank (effective ranks 1002.9 and 1004.3, Frobenius norms 32.7 and 23.6), with flat, slowly-decaying singular value spectra (Figure 16(a)–(b)). The negative mean row cosine means the projection reads the *difference* between current state and anchor rather than their average; we call this the *contrastive projection* property. Directions large in both $x^{(r)}$ and h —stable features unchanged since the prelude—partially cancel; directions large only in $x^{(r)}$ are amplified. This provides a direct mechanistic explanation for why the fixed prelude is helpful: it supplies the reference needed to compute what has changed.

E.9 Attention Entropy: Stable Head Specialisations, Gradual Sharpening

Because identical weights govern every recurrence step, the shared block cannot self-modulate its attention patterns—any change must arise purely from changes in $h^{(r)}$. In Figure 17(a), we capture the full $[B, n_{\text{head}}, S, S]$ attention tensors at each step (where B is batch size and n_{head} the number of attention heads) across all five shared blocks (100 heads total) and measure per-head Shannon entropy and diagonal mass.

Aggregate trend. Mean attention entropy declines from 3.00 nats at step 1 to 2.77 at step 4, then *rebounds* to 2.82 at steps 5–6 (Figure 17(b)). This non-monotone profile mirrors the two-phase CKA structure from Section E.4: steps 1–4 are the refinement phase (lower entropy, more peaked patterns), while the rebound at steps 5–6 aligns with the final convergence plateau where small corrective updates scan broadly. Mean diagonal mass peaks at step 4 (0.108) then drops to 0.092 at step 6.

Head role stability. The full 6×100 entropy heatmap (Figure 17(a)) makes the dominant pattern immediately visible: individual heads retain their character across all six steps. Of 100 heads, 2 are local/sharp (entropy < 1.0), 3 are previous-token (prev-mass > 0.35), 2 are self-attention (diagonal mass > 0.35), 7 are broad broadcast (entropy > 4.0), and 86 are general. Here, *prev-mass* denotes the fraction of total attention weight placed on the immediately preceding token position (i.e. diagonal offset -1 of the attention matrix), averaged over the sequence. A head with prev-mass > 0.35 predominantly attends to the token immediately before each query position. The clearest specialist—Block 0, Head 14—has entropy 0.298 and prev-mass 0.878 at step $r = 1$,

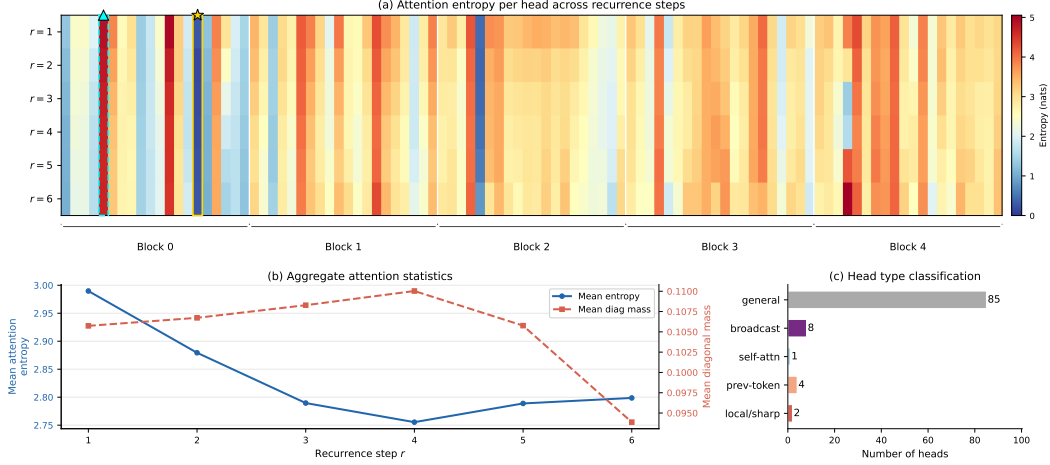


Figure 17: **Attention entropy across recurrence steps.** (a) Full 6×100 entropy heatmap; white lines separate blocks. $\star$: Block 0 Head 14 (previous-token specialist, entropy 0.30). $\triangle$: Block 0 Head 4 (broadcast head, entropy 4.87). Head roles are highly stable. (b) Mean entropy (blue) and diagonal mass (orange) per step; non-monotone entropy profile mirrors the two-phase CKA structure. (c) Head type classification across all 100 heads.

Takeaway

Head specialisations are locked in by weight and do not adapt to recurrence depth. What changes across steps is only the magnitude of each head’s contribution, modulated by the gated update to the hidden state.

E.10 Recurrence Preferentially Serves Two Token Regimes

We split tokens into four BPE frequency bands: byte/punctuation (IDs 0–255, $n = 5,763$), common (IDs 256–2,000, $n = 15,789$), medium (IDs 2,001–10,000, $n = 6,778$), and rare (IDs $> 10,000$, $n = 4,438$), where n refers to the total occurrences of corresponding token id ranges in the validation set. Figure 18(a) shows that all four follow the familiar front-loaded loss-reduction profile, but at very different absolute scales: byte/punctuation tokens improve by just 1.92 nats while rare tokens improve by 3.39 nats.

Figure 18(b) plots the per-step relative improvement $\Delta_r^{\text{rel}} = (L^{(r-1)} - L^{(r)})/L^{(r-1)}$, i.e. the fraction of the *preceding step’s* loss resolved at each step. At step $r = 1$, byte/punctuation tokens already resolve 49% of their previous loss, compared to 23–28% for the other bands—recurrence is disproportionately productive for easy tokens first. Gains diminish uniformly beyond step 3 ($< 4\%$ for all bands), consistent with the fixed-point convergence of Section E.6. Cumulatively over all six steps, byte/punctuation achieves 67% recovery of its initial loss versus 42% for rare tokens. We call these two roles *completion* (sharpening already-likely token predictions) and *lexical narrowing* (progressively concentrating probability mass for hard tokens). Both operate simultaneously in the same forward pass, possible because the elementwise gate allocates computation per-position rather than globally—no explicit routing mechanism is required.

E.11 Unified Summary

The ten analyses above converge on a coherent computational and mechanistic account of how GATED RECURRENT TRANSFORMER’s shared block achieves its parameter efficiency.

Behaviourally, the model operates in two regimes separated by a representational phase transition between steps 1 and 2. In the *early regime* (steps 1–2), the gate is relatively open (≈ 0.82 mean, large update norms, high proposal cosine entropy), the hidden state undergoes a rapid restructuring that resolves $\sim 77\%$ of predictive uncertainty, and CKA confirms a sharp departure from the prelude representation. In the *late regime* (steps 3–6), the gate progressively closes, updates shrink and

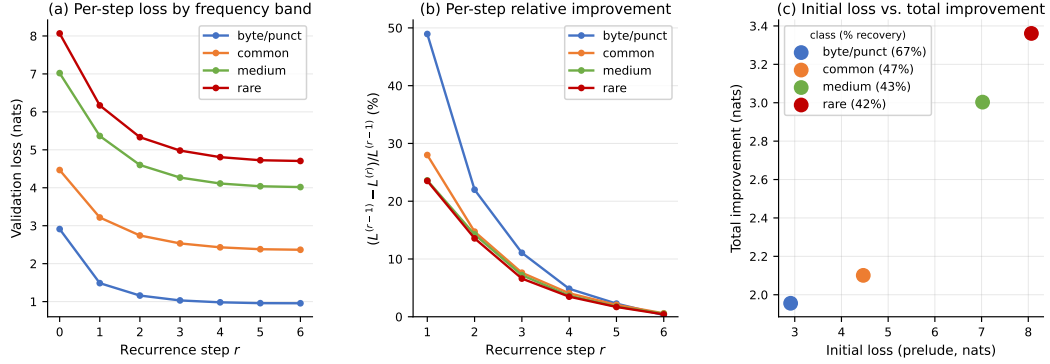


Figure 18: **Token frequency class breakdown.** (a) Per-step loss by BPE frequency band; rare tokens maintain persistently higher loss throughout. (b) Per-step relative improvement $(L^{(r-1)} - L^{(r)}) / L^{(r-1)}$ per frequency class; the first recurrence step alone resolves 47% of byte/punctuation loss vs. 23% for rare tokens—easy tokens converge fastest, hard tokens accumulate improvements gradually. (c) Initial loss vs. total improvement; the two axes are near-orthogonal, consistent with dual completion/narrowing roles.

align, the representation approaches a fixed point, and the attention patterns partially rebroadcast—consistent with targeted correction rather than coarse restructuring.

Mechanistically, four findings explain *why* this behaviour emerges from the learned weights. The *gate weight asymmetry* (W_x leading singular values 44% larger than W_h) primes the gate to respond more strongly to changes in the hidden state than to the anchor, an advantage that is dynamically amplified as $x^{(r)}$ departs from h across steps. The *contrastive projection* (mean row cosine -0.19) ensures the shared block receives an input that emphasises what has changed since the prelude, giving each step access to a difference signal rather than a raw state. The *head specialisations* are fixed by weight—the same 14 previous-token heads, 8 broadcast heads, and 11 self-attention heads appear at every step—so the block applies the same functional program each time; the gate, not the heads, decides how much of each step’s output to incorporate. Finally, the *token frequency breakdown* reveals that a single forward pass simultaneously performs completion for common tokens and lexical narrowing for rare ones, made possible because the elementwise gate implicitly routes computation per-position without any explicit routing mechanism.